Artificial Intelligence

Topic:
Gradient Descent

notes.arpanadhikari7.com.np

By: Ashok Basnet

Available on notes.arpanadhikari7.com.np

01

Gradient Descent

notes.arpanadhikari7.com.np

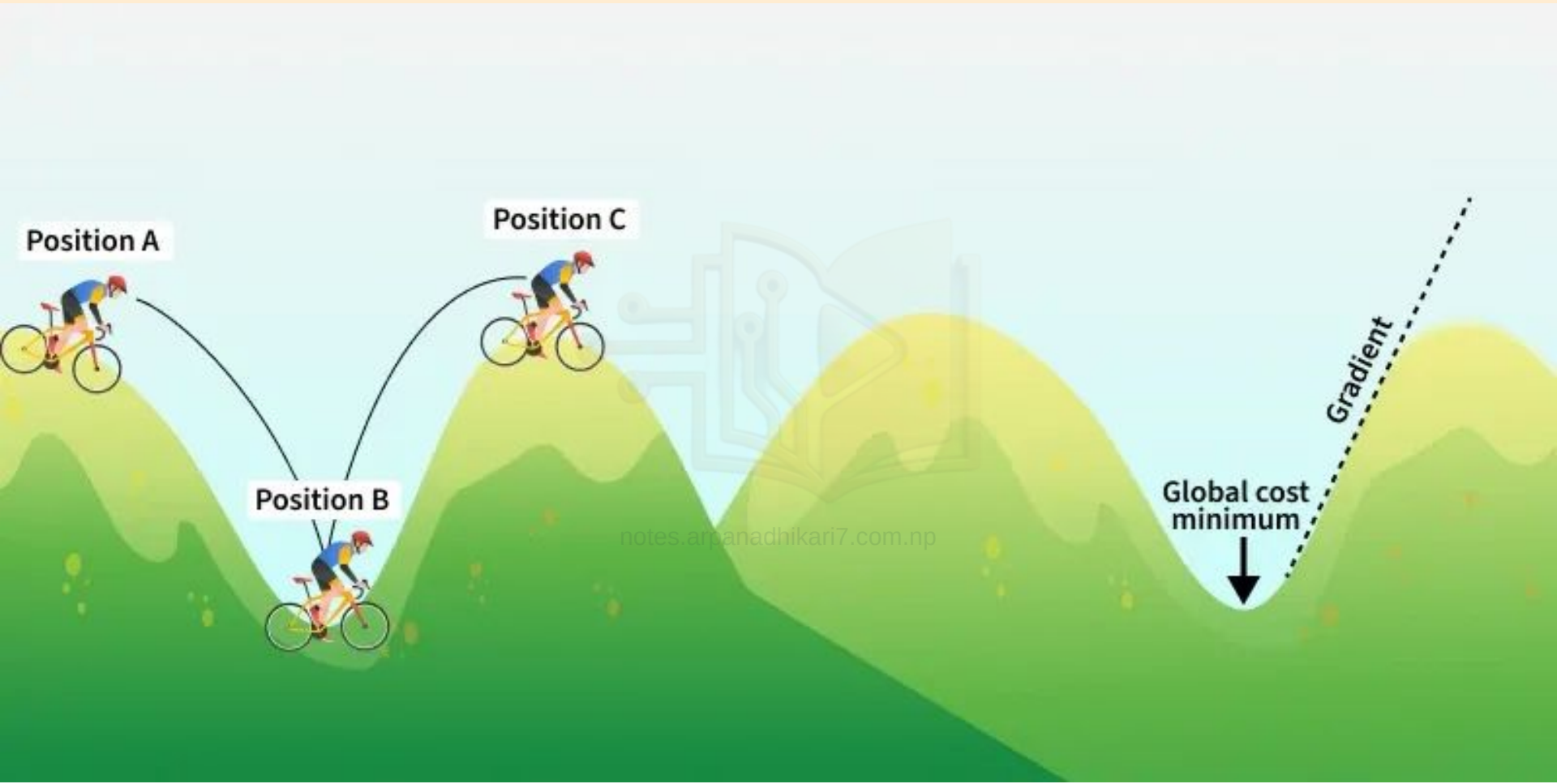
Gradient Descent

Gradient Descent is an iterative optimization algorithm used to minimize a cost function by adjusting model parameters in the direction of the steepest descent of the function’s gradient.

In simple terms, it finds the optimal values of weights and biases by gradually reducing the error between predicted and actual outputs.

Imagine you're at the top of a hill and your goal is to find the lowest point in the valley. You can't see the entire valley from the top, but you can feel the slope under your feet.

Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Gradient Descent

1. **Start at the Top:** You begin at the top of the hill (this is like starting with random guesses for the model's parameters).
2. **Feel the Slope:** You look around to find out which direction the ground is sloping down. This is like calculating the gradient, which tells you the steepest way downhill.
3. **Take a Step Down:** Move in the direction where the slope is steepest (this is adjusting the model's parameters). The bigger the slope, the bigger the step you take.
4. **Repeat:** You keep repeating the process — feeling the slope and moving downhill — until you reach the bottom of the valley (this is when the model has learned and minimized the error).

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Gradient Descent

The key idea is that, just like walking down a hill, Gradient Descent moves towards the "bottom" or minimum of the loss function, which represents the error in predictions.

Moving in opposite direction of the gradient allows the algorithm to gradually descend towards lower values of the function and eventually reaching to the minimum of the function.

These gradients guide the updates ensuring convergence towards the optimal parameter values. Gradual steps used in descent is done by defining **learning rate.**

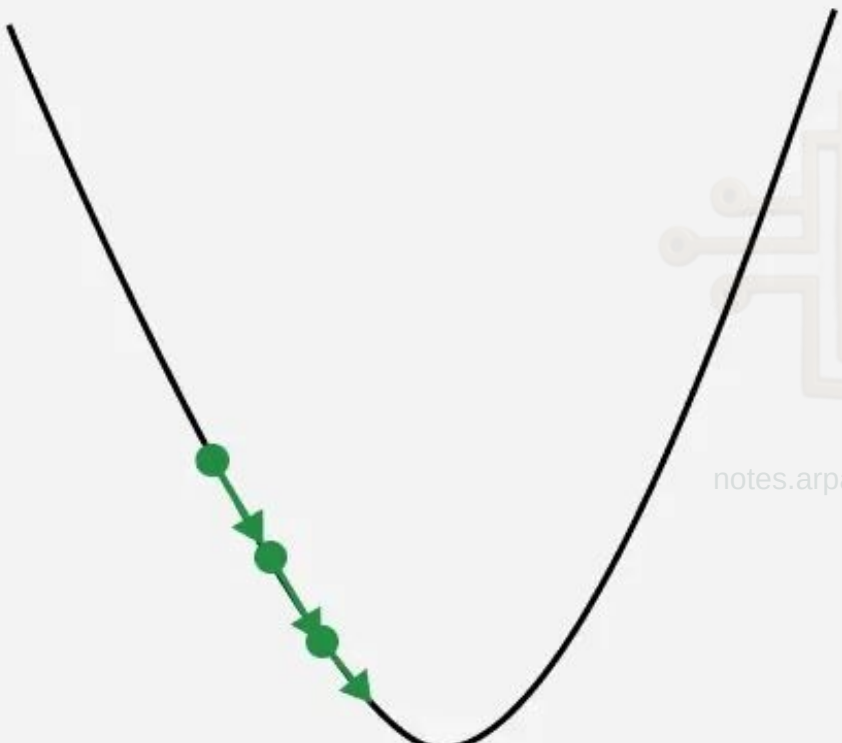
Available on notes.arpanadhikari7.com.np

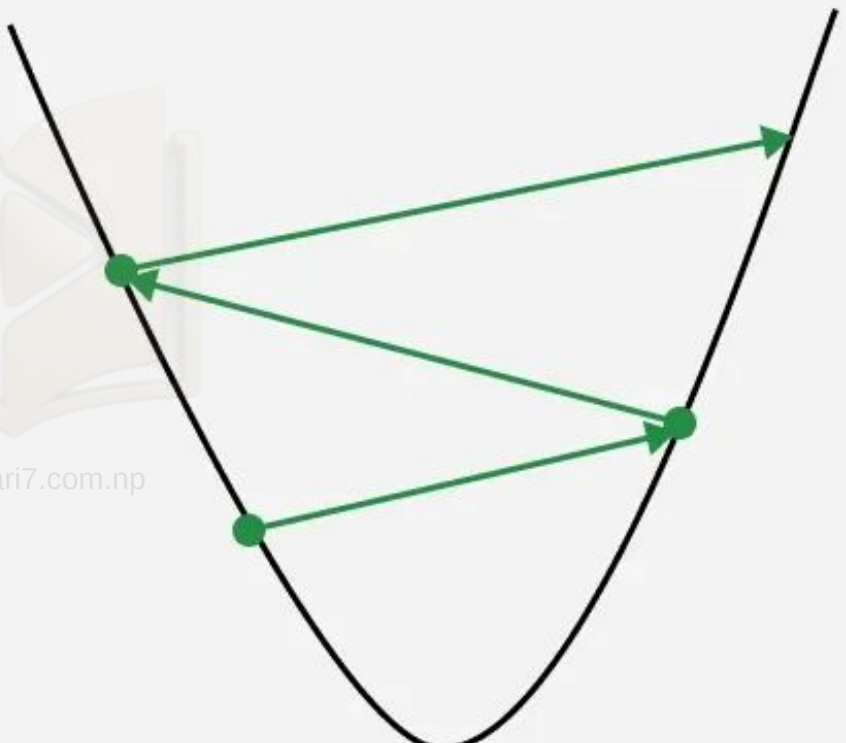
Learning Rate

Learning rate is a important hyperparameter in gradient descent that controls how big or small the steps should be when going downwards in gradient for updating models parameters.

It is essential to determines how quickly or slowly the algorithm converges toward minimum of cost function.

notes.arpanadhikari7.com.np





notes.arpanadhikari7.com.np

Learning Rate

If Learning rate is too small:

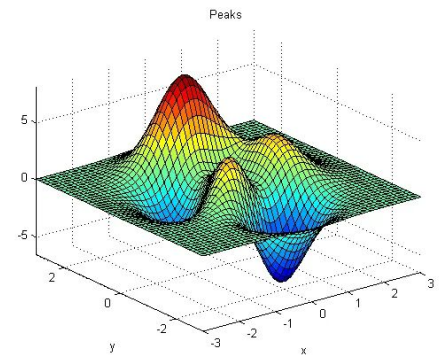
- The algorithm will take tiny steps during iteration and converge very slowly.
- This can significantly increases training time and computational cost especially for large datasets.

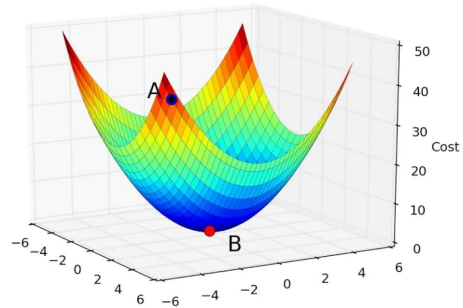
If Learning rate is too big:

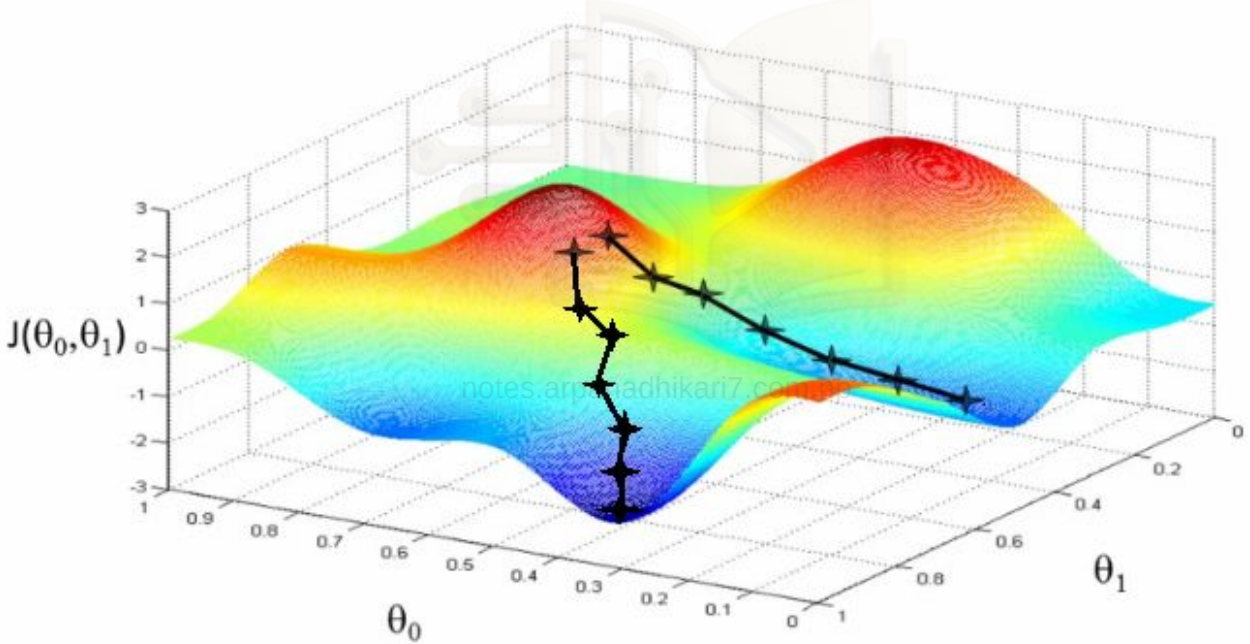
- The algorithm may take huge steps leading overshooting the minimum of cost function without settling.
- It fail to converge causing the algorithm to oscillate.
- This process is termed as exploding gradient problem.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np







Available on notes.arpanadhikari7.com.np

Learning Rate

To address these problems we have some technique that can be used:

- Weights Regularizations:
    - The initialization of weights can be adjusted to ensure that they are in an appropriate range.
    - Using a different activation function such as the Rectified Linear Unit (ReLU) can help us to mitigate the vanishing gradient problem.
- Gradient clipping:
    - Restrict the gradients to a predefined range to prevent them from becoming excessively large or small.

notes.arpanadhikari7.com.np

Learning Rate

- Batch normalization:
  - It can also help to address these problems by normalizing the input of each layer to prevent activation function from saturating and hence reducing vanishing and exploding gradient problems.

Choosing right learning rate can leads to fast and stable convergence improving the efficiency of the training process but sometimes vanishing and exploding gradient problem is unavoidable.

notes.arpanadhikari7.com.np

Linear Regression

Gradient descent minimizes the Mean Squared Error (MSE) which serves as the loss function to find the best-fit line.

Gradient Descent is used to iteratively update the weights (coefficients) and bias by computing the gradient of the MSE with respect to these parameters.

Since MSE is a convex function gradient descent guarantees convergence to the global minimum if the learning rate is appropriately chosen.

notes.arpanadhikari7.com.np

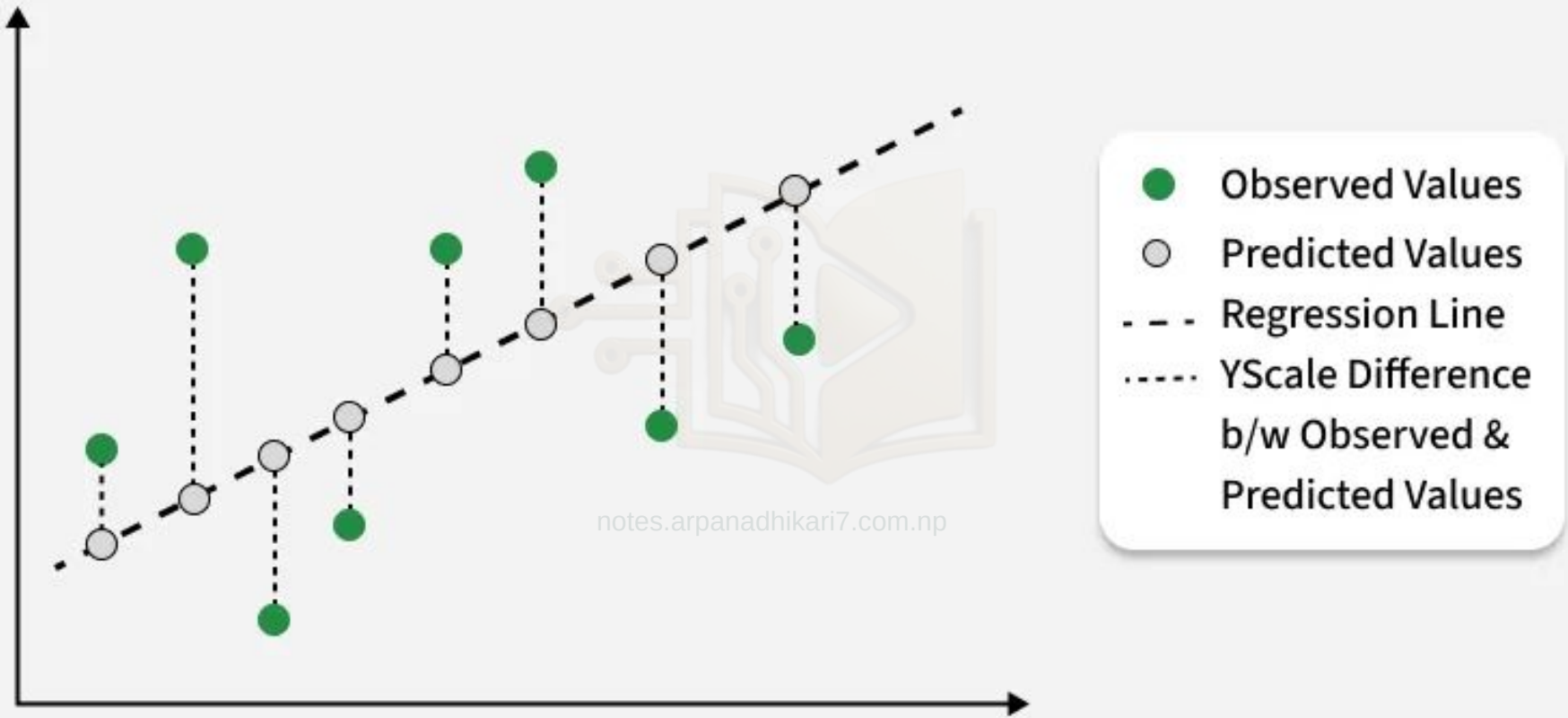
Mean Squared Error

Mean Squared Error (MSE) is a fundamental concept in statistics and machine learning, playing a crucial role in assessing the accuracy of predictive models.

- The MSE value provides a way to analyze the accuracy of the model.
- It measures the average squared difference between predicted values and the actual values in the dataset.
- It is calculated by taking the average of the squared residuals, where the residual is the difference between the predicted value and the actual value for each data point.

notes.arpanadhikari7.com.np

$$\text{Mean Squared Error} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$



notes.arpanadhikari7.com.np

Linear Regression

For each iteration: The algorithm computes the gradient of the MSE with respect to the weights and biases.

Updating weights and biases iteratively to maximize the likelihood of the correct classification:

$$w = w - \alpha \cdot \frac{\partial J(w,b)}{\partial w}, \quad b = b - \alpha \cdot \frac{\partial J(w,b)}{\partial b}$$

notes.arpanadhikari7.com.np

The formula is the parameter update rule for gradient descent, which adjusts the weights w and biases b to minimize a cost function.

This process iteratively adjusts the line's slope and intercept to minimize the error.

Available on notes.arpanadhikari7.com.np

Mathematics Behind Gradient Descent

For simplicity let's consider a linear regression model with a single input feature x and target y.

The loss function (or cost function) for a single data point is defined as the Mean Squared Error (MSE):

$$J(w,b) = \frac{1}{n}\sum_{i=1}^{n}(y_p - y)^2$$

Here:

- $y_p = x \cdot w + b$: The predicted value.
- w: Weight (slope of the line).
- b: Bias (intercept).
- n: Number of data points.

Available on notes.arpanadhikari7.com.np

Gradient of J(w,b) with respect to w

$$
\begin{aligned}
J'_w &= \frac{\partial J(w,b)}{\partial w} \\
&= \frac{\partial}{\partial w}\left[\frac{1}{n}(y_p - y)^2\right] \\
&= \frac{2(y_p - y)}{n}\frac{\partial}{\partial w}\left[(y_p - y)\right] \\
&= \frac{2(y_p - y)}{n}\frac{\partial}{\partial w}\left[((xW^T + b) - y)\right] \\
&= \frac{2(y_p - y)}{n}\left[\frac{\partial(xW^T + b)}{\partial w} - \frac{\partial(y)}{\partial w}\right] \\
&= \frac{2(y_p - y)}{n}\left[x - 0\right] \\
&= \frac{1}{n}(y_p - y)[2x]
\end{aligned}
$$

Gradient of J(w,b) with respect to b

$$
\begin{aligned}
J'_b &= \frac{\partial J(w,b)}{\partial b} \\
&= \frac{\partial}{\partial b}\left[\frac{1}{n}(y_p - y)^2\right] \\
&= \frac{2(y_p - y)}{n}\frac{\partial}{\partial b}\left[(y_p - y)\right] \\
&= \frac{2(y_p - y)}{n}\frac{\partial}{\partial b}\left[((xW^T + b) - y)\right] \\
&= \frac{2(y_p - y)}{n}\left[\frac{\partial(xW^T + b)}{\partial b} - \frac{\partial(y)}{\partial b}\right] \\
&= \frac{2(y_p - y)}{n}\left[1 - 0\right] \\
&= \frac{1}{n}(y_p - y)[2]
\end{aligned}
$$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

$$w = w - \gamma \cdot \frac{\partial J(w,b)}{\partial w}$$

$$w = w - \gamma \nabla J(w,,b)$$
$$b = b - \gamma \nabla J(w,,b)$$

notes.arpanadhikari7.com.np

Gradient Descent

1. First initialize the parameters of the model randomly
2. Compute the gradient of the cost function with respect to each parameter. It involves making partial differentiation of cost function with respect to the parameters.
3. Update the parameters of the model by taking steps in the opposite direction of the model. Here we choose a hyperparameter learning rate which is denoted by γ. It helps in deciding the step size of the gradient.
4. Repeat steps 2 and 3 iteratively to get the best parameter for the defined model.

Available on notes.arpanadhikari7.com.np

02

Backtracking

Available on notes.arpanadhikari7.com.np

Backtracking Algorithm

Backtracking algorithms are like problem-solving strategies that help explore different options to find the best solution.

They work by trying out different paths and if one doesn't work, they backtrack and try another until they find the right one.

It's like solving a puzzle by testing different pieces until they fit together perfectly.

Backtracking is a problem-solving algorithmic technique that involves finding a solution incrementally by trying different options and undoing them if they lead to a dead end.

Available on notes.arpanadhikari7.com.np

Backtracking Algorithm

A backtracking algorithm works by recursively exploring all possible solutions to a problem.

It starts by choosing an initial solution, and then it explores all possible extensions of that solution.

If an extension leads to a solution, the algorithm returns that solution.

If an extension does not lead to a solution, the algorithm backtracks to the previous solution and tries a different extension.

notes.arpanadhikari7.com.np

Backtracking Algorithm

The following is a general outline of how a backtracking algorithm works:

1. Choose an initial solution.
2. Explore all possible extensions of the current solution.
3. If an extension leads to a solution, return that solution.
4. If an extension does not lead to a solution, backtrack to the previous solution and try a different extension.
5. Repeat steps 2-4 until all possible solutions have been explored.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

```
function backtracking(state):
    if state is a solution:
        return state

    for choice in all possible choices:
        if choice is valid:
            make choice
            result = backtracking(state with choice)
            if result is not failure:
                return result
            undo choice
    return failure
```

notes.arpanadhikari7.com.np

N-Queens problem

1. Initialize an empty chessboard of size N x N.
2. Start placing queens on the chessboard, one column at a time.
3. Before placing a queen in a column, check if it is safe to place the queen in that position.
4. If the queen can be placed safely, proceed to the next column recursively.
5. If no safe position is found, backtrack to the previous column and try the next position.
6. Repeat the process until all queens are placed on the chessboard or all possibilities are explored.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

03

Min Max Algorithm

Available on notes.arpanadhikari7.com.np

Minimax Algorithm

Minimax is a kind of backtracking algorithm that is used in decision making and game theory to find the optimal move for a player, assuming that your opponent also plays optimally.

It is widely used in two player turn-based games such as Tic-Tac-Toe, Backgammon, Mancala, Chess, etc.

In Minimax the two players are called **maximizer and minimizer.**

The maximizer tries to get the highest score possible while the minimizer tries to do the opposite and get the lowest score possible.

Mini-Max algorithm is a **decision-making algorithm** used in artificial intelligence, particularly in game theory and computer games.

Available on notes.arpanadhikari7.com.np

Working of Min-Max Process

Players Involved

Maximizing Player (Max):

- Aims to maximize their score or utility value.
- Chooses the move that leads to the highest possible utility value, assuming the opponent will play optimally.

Minimizing Player (Min):

- Aims to minimize the maximizer's score or utility value.
- Selects the move that results in the lowest possible utility value for the maximizer, assuming the opponent will play optimally.

Available on notes.arpanadhikari7.com.np

Working of Min-Max Process

The Min-Max algorithm involves several key steps, executed recursively until the optimal move is determined. Here is a step-by-step breakdown:

Step 1: Generate the Game Tree

- Objective: Create a tree structure representing all possible moves from the current game state.
- Details: Each node represents a game state, and each edge represents a possible move.

Step 2: Evaluate Terminal States

- Objective: Assign utility values to the terminal nodes of the game tree.
- Details: These values represent the outcome of the game (win, lose, or draw).

notes.arpanadhikari7.com.np

Working of Min-Max Process

Step 3: Propagate Utility Values Upwards

- Objective: Starting from the terminal nodes, propagate the utility values upwards through the tree.
- Details: For each non-terminal node:
  - If it's the maximizing player's turn, select the maximum value from the child nodes.
  - If it’s the minimizing player's turn, select the minimum value from the child nodes.

Step 4: Select Optimal Move

- Objective: At the root of the game tree, the maximizing player selects the move that leads to the highest utility value.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

```
def minmax(state, depth, maximizing_player):
    if is_terminal(state) or depth == 0:
        return utility(state)

    if maximizing_player:
        max_eval = -infinity
        for action in actions(state):
            eval = minmax(result(state, action), depth - 1, False)
            max_eval = max(max_eval, eval)
        return max_eval
    else:
        min_eval = infinity
        for action in actions(state):
            eval = minmax(result(state, action), depth - 1, True)
            min_eval = min(min_eval, eval)
        return min_eval
```

notes.arpanadhikari7.com.np

Alpha-Beta Pruning Optimization

Alpha-beta pruning enhances the Min-Max algorithm by eliminating branches that do not affect the final decision.

The key formulas for alpha-beta pruning are:

- **Alpha (α):** The best value that the maximizing player can guarantee so far.
- **Beta (β):** The best value that the minimizing player can guarantee so far.

During the search:

- If $\alpha \geq \beta$, prune the remaining branches.

notes.arpanadhikari7.com.np

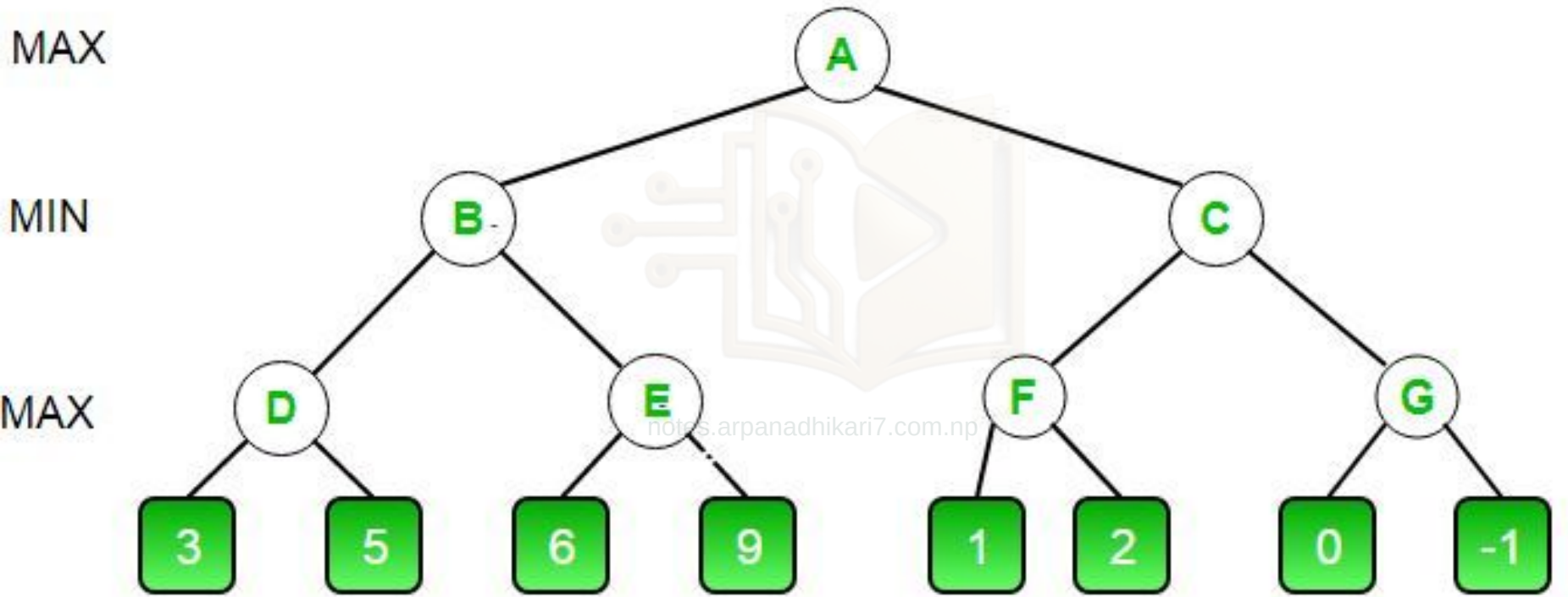
```
def alpha_beta_minmax(state, depth, alpha, beta, maximizing_player):
    if is_terminal(state) or depth == 0:
        return utility(state)

    if maximizing_player:
        max_eval = -infinity
        for action in actions(state):
            eval = alpha_beta_minmax(result(state, action), depth - 1,
alpha, beta, False)
            max_eval = max(max_eval, eval)
            alpha = max(alpha, eval)
            if beta <= alpha:
                break  # Beta cut-off
        return max_eval
```

notes.arpanadhikari7.com.np

```
    else:
        min_eval = infinity
        for action in actions(state):
            eval = alpha_beta_minmax(result(state, action), depth - 1,
alpha, beta, True)
            min_eval = min(min_eval, eval)
            beta = min(beta, eval)
            if beta <= alpha:
                break  # Alpha cut-off
        return min_eval
```

Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Alpha-Beta Pruning Optimization

The initial call starts from A.

- The value of alpha here is -INFINITY and the value of beta is +INFINITY. These values are passed down to subsequent nodes in the tree.
- At A the maximizer must choose max of B and C, so A calls B first
- At B it the minimizer must choose min of D and E and hence calls D first.
- At D, it looks at its left child which is a leaf node. This node returns a value of 3. Now the value of alpha at D is max(-INF, 3) which is 3.
- To decide whether its worth looking at its right node or not, it checks the condition beta<=alpha. This is false since beta = +INF and alpha = 3. So it continues the search.

Available on notes.arpanadhikari7.com.np

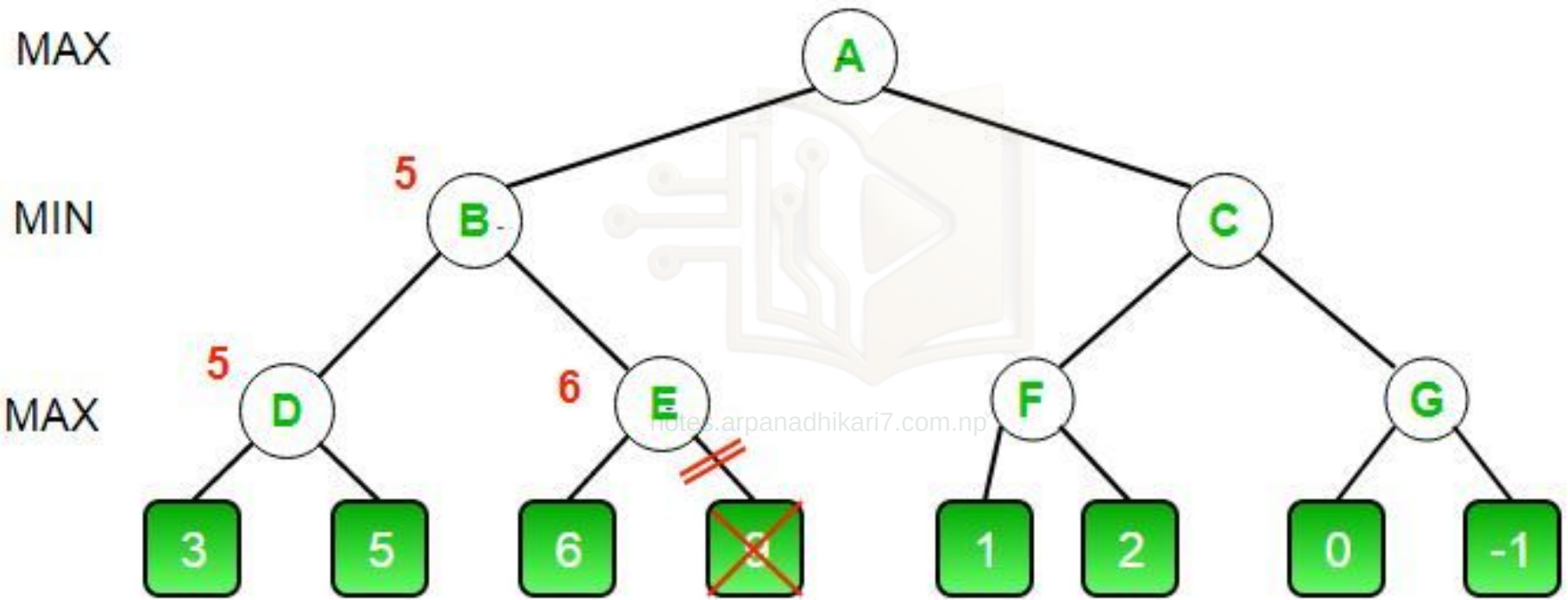
Alpha-Beta Pruning Optimization

The initial call starts from A.

- The value of alpha here is -INFINITY and the value of beta is +INFINITY. These values are passed down to subsequent nodes in the tree.
- At A the maximizer must choose max of B and C, so A calls B first
- At B it the minimizer must choose min of D and E and hence calls D first.
- At D, it looks at its left child which is a leaf node. This node returns a value of 3. Now the value of alpha at D is max(-INF, 3) which is 3.
- To decide whether its worth looking at its right node or not, it checks the condition beta<=alpha. This is false since beta = +INF and alpha = 3. So it continues the search.

Available on notes.arpanadhikari7.com.np

D now looks at its right child which returns a value of 5.At D, alpha = max(3, 5) which is 5. Now the value of node D is 5



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Alpha-Beta Pruning Optimization

- D returns a value of 5 to B. At B, beta = min(+INF, 5) which is 5.
- The minimizer is now guaranteed a value of 5 or lesser.
- B now calls E to see if he can get a lower value than 5.
- At E the values of alpha and beta is not -INF and +INF but instead -INF and 5 respectively, because the value of beta was changed at B and that is what B passed down to E
- Now E looks at its left child which is 6.
    - At E, alpha = max(-INF, 6) which is 6. Here the condition becomes true. beta is 5 and alpha is 6. So beta<=alpha is true. Hence it breaks and E returns 6 to B

Available on notes.arpanadhikari7.com.np

Alpha-Beta Pruning Optimization

- Note how it did not matter what the value of E's right child is.
- It could have been +INF or -INF, it still wouldn't matter, We never even had to look at it because the minimizer was guaranteed a value of 5 or lesser.
- So as soon as the maximizer saw the 6 he knew the minimizer would never come this way because he can get a 5 on the left side of B.
- This way we didn't have to look at that 9 and hence saved computation time.
- E returns a value of 6 to B. At B, beta = min(5, 6) which is 5.The value of node B is also 5

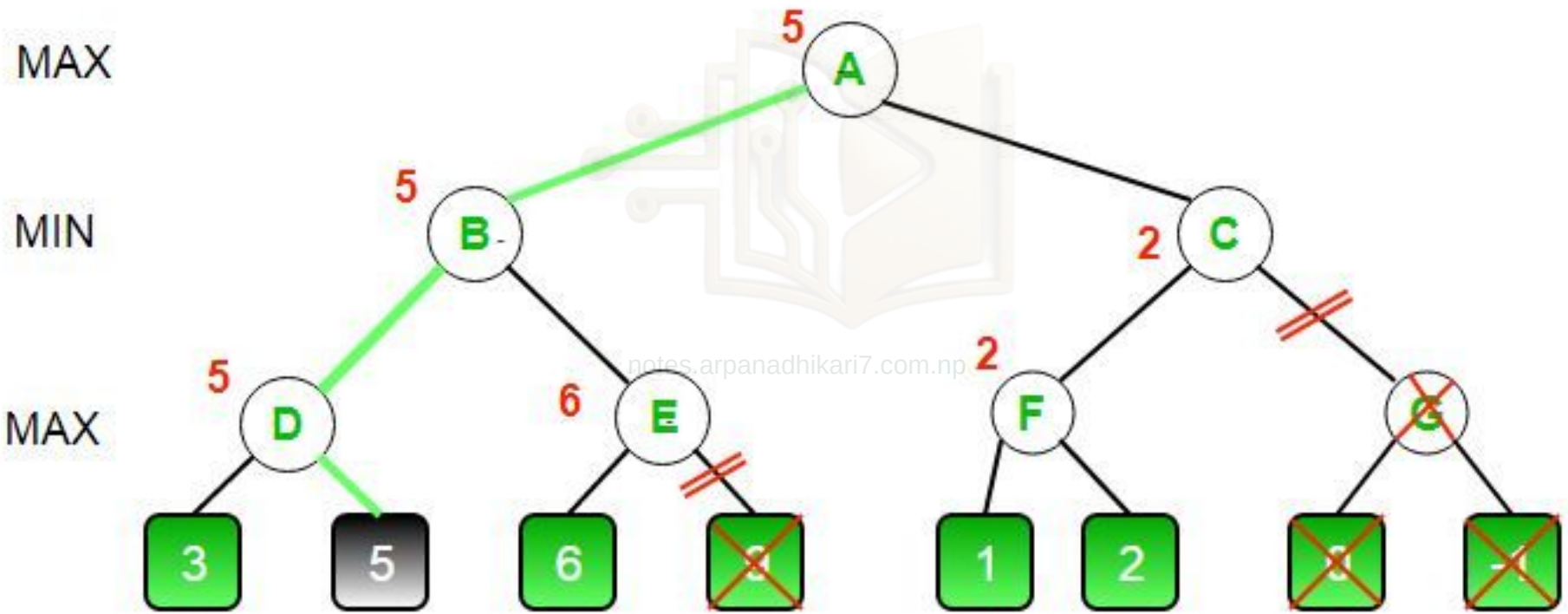
notes.arpanadhikari7.com.np

Alpha-Beta Pruning Optimization

- B returns 5 to A. At A, alpha = max(-INF, 5) which is 5.
- Now the maximizer is guaranteed a value of 5 or greater.
- A now calls C to see if it can get a higher value than 5.
- At C, alpha = 5 and beta = +INF. C calls F
- At F, alpha = 5 and beta = +INF.
- F looks at its left child which is a 1. alpha = max(5, 1) which is still 5.
- F looks at its right child which is a 2. Hence the best value of this node is 2.
- Alpha still remains 5

Available on notes.arpanadhikari7.com.np

F returns a value of 2 to C. At C, beta = min(+INF, 2).
The condition beta <= alpha becomes true as beta = 2 and alpha = 5.
So it breaks and it does not even have to compute the entire sub-tree of G.



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Alpha-Beta Pruning Optimization

The intuition behind this break-off is that, at C the minimizer was guaranteed a value of 2 or lesser.

But the maximizer was already guaranteed a value of 5 if he choose B.

So why would the maximizer ever choose C and get a value less than 2 ?

- We can see that it did not matter what those last 2 values were.
- We also saved a lot of computation by skipping a whole sub-tree.
- C now returns a value of 2 to A.
- Therefore the best value at A is max(5, 2) which is a 5.

Hence the optimal value that the maximizer can get is 5

notes.arpanadhikari7.com.np

Thank you



Ready for what's next?

Let's talk

Name: Ashok Basnet
Email: ashok.basnet@ncit.edu.np

Available on notes.arpanadhikari7.com.np